

OPERATIONS RECORD · PRIVACY SUITE

Getting Portmaster Working on Orange Pi 5 Plus

How the official x86 installer failed on this ARM board, and how Safing’s published linux_arm64 v1 stack was installed, started, and verified.

Prepared 15 August 2026 on host orangepi5plus. Portmaster is a free, open-source application firewall from Safing (safing.io). The paid Safing Privacy Network (SPN) logs in after the core is running. This record is written so the work can be reprinted, handed to another operator, or used to rebuild the install.

1. Machine as found

Before any install, the box was checked for packages, binaries, systemd units, and leftover downloads. Portmaster was not installed. A discarded official v2 Debian package was already in the user Trash — evidence of an earlier attempt with the wrong CPU architecture.

Host	orangepi5plus
Board	Orange Pi 5 Plus · Rockchip RK3588
OS	Orange Pi 1.2.0 Jammy · Ubuntu 22.04.5 LTS
Kernel	6.1.43-rockchip-rk3588 (Linux 5.7+ required; this kernel qualifies)
CPU / dpkg arch	aarch64 · arm64
Desktop	XFCE on DISPLAY=:0.0
Network stack	NetworkManager active · systemd-resolved active
Sudo	User orangepi is in group sudo (password required)

Table 1. Host identity at the time of install.

2. Why the official download would not install

Safing’s current desktop product is Portmaster v2. The public download page only ships 64-bit Intel/AMD packages. On this board, `uname -m` is `aarch64`. An `amd64 .deb` cannot execute here.

What we tried	Result	Why
Official v2 Ubuntu package Portmaster_2.2.1_amd64.deb	HTTP 200, but wrong CPU	Architecture: amd64. Will not install on arm64.
Leftover local file in Trash Portmaster_2.1.19_amd64.deb	Present, unused	Same architecture mismatch from an earlier download.
v2 ARM URL linux_arm64/packages/...arm64.deb	HTTP 404	Safing does not publish a v2 .deb for ARM.
GitHub issue #2038 ARM64 / AARCH64 (v1 and v2)	Closed as not planned	Confirms official ARM desktop v2 is not on the roadmap.
install.sh auto-detect	Would reject this board	Script matches <code>uname -m</code> to <code>x86_64</code> or <code>arm64</code> . Linux reports <code>aarch64</code> .

Table 2. Dead ends on the official v2 path.

The installer script does contain an `arm64` case, but it never sees it on Linux ARM because the kernel reports `aarch64`, not `arm64`. Passing `--arch arm64` is required.

3. What actually exists for ARM64

Safing's update index still publishes a complete Portmaster v1 `linux_arm64` set. Those binaries were downloaded and checked with `file` before install: they are real AArch64 ELF executables, not x86 binaries in disguise. Desktop libraries needed by the Electron UI were already on the board (GTK 3, WebKitGTK, Ayatana AppIndicator).

Component	Version	Verified as
portmaster-start	1.6.0	ELF 64-bit LSB, ARM aarch64, Go 1.21.2
portmaster-core	1.6.10	ELF 64-bit LSB, ARM aarch64, statically linked
portmaster-app	0.2.8	ELF 64-bit LSB pie, ARM aarch64 (Electron)
portmaster-notifier	0.3.6	linux_arm64 signed update
Filter / GeolIP intel	2025-2026 lists	Downloaded during first update

Table 3. ARM64 artifacts that were installed.

4. Install that worked

The official Safing installer script was used — not a third-party package and not a from-source build. Root was obtained with `pkexec` so a desktop password dialog could appear on XFCE. Architecture was forced because auto-detect would have failed.

Official installer, ARM override:

```
curl -fsSL https://updates.safing.io/latest/linux_all/packages/install.sh -o /tmp/portmaster-install.sh
pkexec bash /tmp/portmaster-install.sh --arch arm64
```

The script wrote files to `/opt/safing/portmaster`, installed icons, added desktop launchers, and enabled `portmaster.service` for boot. It then pulled every signed module (core, app, notifier, intel lists). That download ran several minutes and finished with:

```
Portmaster is now installed. Please restart your device to start Portmaster
```

A full reboot was not required to bring the core up. The service was started immediately so SPN login could happen the same afternoon:

```
pkexec systemctl daemon-reload && pkexec systemctl start portmaster.service
/opt/safing/portmaster/portmaster-start app --data=/opt/safing/portmaster
```

5. State after install (verified 15 Aug 2026, 14:43 MST)

Service	portmaster.service · active (running) · enabled at boot
Core PID	root · /opt/safing/portmaster/updates/linux_arm64/core/portmaster-core_v1-6-10
App	User orangeipi · Portmaster UI running on XFCE
Notifier	User session notifier running
Local API	127.0.0.1:817 (UI talks to the core here)
Install root	/opt/safing/portmaster
Launchers	/usr/share/applications/portmaster.desktop
Network check	DNS and ping 1.1.1.1 succeeded after start

Table 4. Live state after the first successful start.

6. What did not come along for the ride

Two limitations are real on this board. Neither stopped the firewall from starting. Both should be expected if someone reads the journal and thinks the install failed.

Limitation	What the log says	Practical effect
No Portmaster v2 on ARM	linux_arm64 v2 .deb is 404; stable.v3.json lists linux_amd64 only	This install is v1. SPN login still happens in the app. UI is the older Electron shell.
Vendor kernel has no BTF	no BTF found for kernel version 6.1.43-rockchip-rk3588: not supported	eBPF extras (bandwidth stats, CO-RE probes) give up. Interception falls back to nfqueue, which is the classic Linux path and is enough for the firewall.
Filter lists load slowly	intel/filterlists: not searching ... because filterlists not loaded	Normal on first start while lists unpack. Domain blocklists become accurate after a few minutes.
Installer wants a reboot	a reboot may be required (unit file upgraded on disk)	Service was started without reboot and stayed healthy. A later reboot still settles DNS integration cleanly.

Table 5. Known caveats on this Rockchip kernel.

Do not paste Safing account passwords into chat.

Portmaster itself is free and needed no login to install. SPN (the paid privacy network) is unlocked inside the running app: open Portmaster → SPN / Privacy Network → sign in with the Safing account. Credentials stay on this machine.

7. How to use it from here

Open the app

Applications menu → Portmaster, or run the command above. The notifier sits in the tray and asks about new connections.

Sign in to SPN

In the app, open the SPN panel and enter the Safing account. The install is already done; login only turns on the privacy network.

Watch this box's listeners

This Orange Pi was already serving SSH and several other sockets when Portmaster came up. If something suddenly cannot connect, open the app and allow that process or port rather than assuming the service died.

Port	Notes
22	SSH on all interfaces. Do not leave this blocked.
817	Portmaster core API, localhost only. Used by the UI.
5050, 30001–30005, 30104	Already listening before install. Prompt/allow in the app if they break.
53 (127.0.0.53)	systemd-resolved. Portmaster may take over DNS after a reboot.

Table 6. Local listeners that matter after the firewall is live.

8. Operator cheat sheet

Task	Command
Is the core up?	<code>systemctl status portmaster.service</code>
Start / stop / restart	<code>sudo systemctl start stop restart portmaster.service</code>
Follow the log	<code>journalctl -u portmaster.service -f</code>
Open the UI	<code>/opt/safing/portmaster/portmaster-start app --data=/opt/safing/portmaster</code>
Manual update	<code>sudo /opt/safing/portmaster/portmaster-start update --data=/opt/safing/portmaster</code>
Clean leftover iptables after a crash	<code>sudo /opt/safing/portmaster/portmaster-start recover-iptables</code>
Uninstall (official script)	<code>sudo bash /tmp/portmaster-install.sh --uninstall</code>
Uninstall and wipe config	<code>sudo bash /tmp/portmaster-install.sh --uninstall --purge</code>

Table 7. Day-to-day commands on this host.

9. Why this counts as a win

The obvious path — download the Ubuntu button on safing.io — is a trap on Orange Pi. The package is beautiful, signed, and completely the wrong ISA. The working path was still official Safing infrastructure: their installer, their `linux_arm64` update tree, their signatures. No qemu, no box64, no rebuilt kernel, no source compile.

At verification time the core had been running under systemd, the UI and notifier were up on the local desktop, the API was listening on `localhost:817`, and the machine still had working DNS and outbound ICMP. That is a complete, boot-persistent Portmaster install on RK3588 ARM64.

Sources used during the work: safing.io/download, wiki.safing.io Portmaster Linux install, updates.safing.io stable.json and stable.v3.json, GitHub [safing/portmaster](https://github.com/safing/portmaster) issues #758 and #2038, and live inspection of this host on 15 August 2026.

Printed for the operator of orangeipi5plus. Rebuild by re-running the two installer commands in section 4 with `--arch arm64`. Do not install the `amd64` .deb from the website on this board.